FastTGM: Efficient Seed-Rooted Temporal Motif Counting with Compressed States

Anonymous Author(s)

Abstract

Temporal motifs, small time-respecting interaction patterns in temporal graphs, are a powerful analytical primitive for characterizing entity behavior. We study seed-rooted temporal motif counting: given a seed vertex, count all temporal motif instances of a given size within a time window, aggregated by motif class into a temporal motif vector. Existing methods scale poorly to motifs with more edges because they track every concrete partial realization separately. We propose FastTGM, a compressed counting framework that replaces occurrence-by-occurrence search with equivalence-class-level counting. FastTGM introduces the Expiring Temporal Frontier Automaton (ETFA), where realized vertices expire once they have no future incident edge, and merges temporally future-equivalent partial realizations into one compressed state carrying a multiplicity counter. FastTGM further incorporates slack-aware completion pruning, which tracks residual structural budget to eliminate states that cannot reach a connected completion within the remaining chronological suffix. Experiments on five public and one industrial temporal graphs show that FastTGM achieves speedups of one to two orders of magnitude at motif sizes 5–6 and is the only method that scales to size 6 on all datasets. An industrial case study further demonstrates that temporal motif features achieve 33%–43% higher PR-AUC than static motif features on a fraud-detection task and enable earlier fraud detection.

1 Introduction

On platforms such as e-commerce marketplaces, social networks, and financial systems, entities interact through timestamped events: money transfers, messages, follows, blocks, and so on [15]. Meaningful behaviors in these settings often manifest not as isolated actions, but as short-lived interaction episodes involving multiple entities in a specific temporal order. For example, a victim transfers money to a fraudster, who blocks the victim shortly after and then quickly forwards the funds to another account. Such a pattern is a *temporal motif* [24, 30, 38]: a small set of events among a few entities, occurring within a time window and in a particular order.

Temporal motifs are more expressive than simpler abstractions. Sequence-based methods [1, 8, 13, 50] model per-entity action histories but miss multi-entity structural relationships. Static graph motifs [2, 7, 34] capture higher-order topology but collapse temporally distinct behaviors into identical patterns. By jointly modeling *who* interacts with *whom*, in what *order*, and within what *time span*, temporal motifs retain temporal-structural signal that neither sequences nor static patterns alone can capture. Figure 1 illustrates this on an industrial temporal graph: static motif-count distributions overlap heavily between risky and benign users, while incorporating temporal order yields substantially more discriminative distributions.

In our setting, we characterize each entity by counting the temporal motif instances rooted at that entity. This gives rise to the *seed-rooted temporal motif counting* problem: given a temporal graph and a designated seed vertex, count all temporal motif instances of a given size rooted at the seed. These counts can then be aggregated by motif class into a *temporal motif vector* (TMV), which serves as a feature representation for downstream pattern discovery, risk assessment, and model training [5]. In industrial deployments, temporal motif counting is the dominant computational bottleneck of such workflows, while downstream mining and evaluation are comparatively lightweight. Improving the scalability of temporal motif counting is therefore the main focus of this paper.

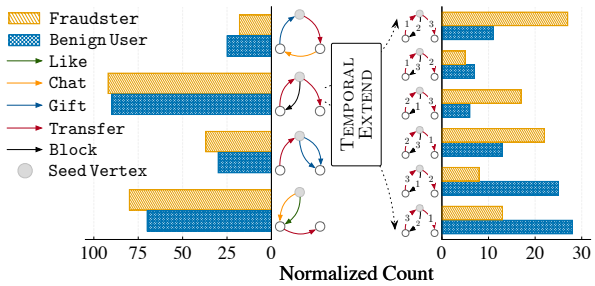


Figure 1: On an industrial temporal interaction graph from ABC (the counterpart of TikTok), static motifs provide limited separability between risky and benign users by frequency, while incorporating temporal order yields substantially more discriminative distributions.

Challenges in scaling temporal motif counting. Traversal-based expansion methods grow motif instances by following incident temporal edges, but the same instance can be reached through multiple expansion orders, and the resulting deduplication overhead grows rapidly with motif size. Query-based temporal matching [32, 33] avoids duplicates within each query but must issue one query per canonical motif pattern; since the number of patterns itself grows combinatorially, this per-query paradigm does not scale when all motif classes must be counted simultaneously. Chronological combinatorial methods eliminate both problems by fixing a chronological edge order and counting all motif classes in one pass. However, they still enumerate every concrete partial realization separately, which leads to two fundamental bottlenecks:

- **(C1) Redundant future-equivalent histories.** As the search progresses through the chronological suffix, realized vertices expire from the remaining future, typically rendering over 90% of the active search states redundant under future equivalence, yet existing methods still track and expand them separately.
- **(C2) Infeasible continuation states.** A partial realization may be locally valid but already impossible to complete into a connected motif within the remaining chronological suffix. Without detecting this early, the search expands an entire futile continuation subspace.

Proposed approach. We propose FastTGM, a compressed counting framework for exact seed-rooted temporal motifs that addresses

both (C1) and (C2). The key departure from all existing temporal motif methods is a shift from occurrence-level search to *equivalence-class-level counting*: instead of expanding each concrete partial realization individually, FastTGM merges future-equivalent histories into one compressed state and counts their continuations once.

To address (C1), FastTGM introduces the *Expiring Temporal Frontier Automaton* (ETFA). Starting from each seed-incident edge, ETFA maps all future-equivalent partial realizations to one compressed counting state carrying a multiplicity counter, so that exact continuation counting is performed once per equivalence class rather than once per concrete history. Each state captures a canonical motif prefix descriptor and the *live frontier* of concrete vertices still visible to the remaining chronological suffix. As vertices expire from the suffix, their concrete identities are safely forgotten while the canonical prefix structure is preserved. We prove that this compression is exact: temporally future-equivalent histories produce identical continuation counts.

To address (C2), FastTGM tracks a residual *slack* budget that measures how much structural freedom remains in the suffix for each counting state. Each take transition consumes a measurable amount of non-bridge budget, so the slack directly quantifies completion feasibility. When the slack becomes small, FastTGM triggers progressively stronger temporal feasibility checks that prune states unable to reach a connected completion.

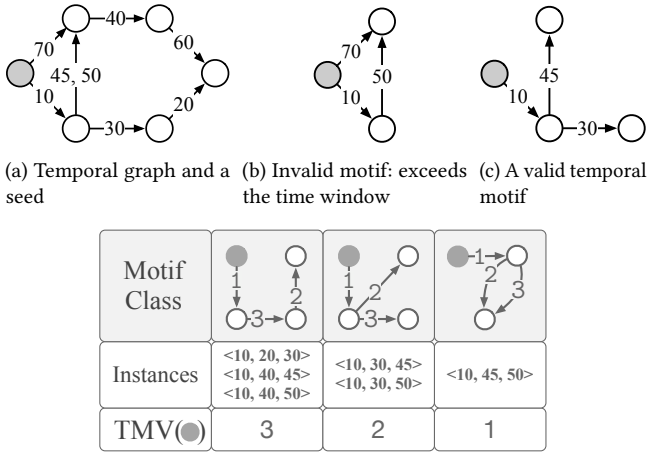
Experiments on five public temporal graphs and one industrial dataset show that FastTGM achieves speedups of one to two orders of magnitude over the strongest baselines at motif sizes $L = 5$–6. At $L = 6$, FastTGM is the only method that completes on all six datasets. We further demonstrate practical utility through an industrial case study on risk-pattern mining: temporal motif features achieve 33%–43% higher PR-AUC than static motif features, with the advantage growing at larger motif sizes. Temporal patterns also detect emerging risks ahead of manual review.

Contributions. We summarize our contributions as follows.

- **Problem formalization.** We formalize seed-rooted temporal motif counting as a general per-seed feature extraction problem, in contrast to prior work that targets global aggregates or fixed small motif sizes [9, 38] (Section 2).
- **Temporal future-equivalent state compression.** We propose ETFA, to our knowledge the first compressed counting framework for temporal motifs. ETFA collapses future-equivalent partial realizations via temporal state sufficiency and processes the chronological suffix in one shared pass with provable exactness (Section 3).
- **Slack-aware completion pruning.** We develop a budget-driven pruning strategy with low-slack temporal certificates and bounded fallback that eliminates infeasible continuation states (Section 4).
- **Extensive evaluation and industrial case study.** We conduct extensive experiments on large-scale graphs, and validate practical utility through a risk-pattern mining case study on an industrial commerce dataset (Sections 5 and 6).

2 Problem Setting

We first introduce the notation for labeled temporal graphs and the seed-rooted temporal motifs studied in this paper. Our definitions



(a) Temporal graph and a seed (b) Invalid motif: exceeds the time window (c) A valid temporal motif

(d) Motif classes and a temporal motif vector

Figure 2: Illustration of a seed-rooted temporal motif under the setting $L = 3$ and $\delta = 50$.

follow the temporal motif literature [38] but are extended to support vertex labels, edge labels, and a distinguished seed vertex.

2.1 Labeled Temporal Graphs

Let Σ_V and Σ_E be finite alphabets of vertex and edge labels, respectively.

Definition 2.1 (Labeled temporal graph)**.** A *labeled temporal graph* is a tuple $\mathcal{T} = (V, E, \lambda_V, \lambda_E, \tau)$, where V is a finite set of vertices, E is a finite multiset of directed temporal edges, $\lambda_V : V \to \Sigma_V$ and $\lambda_E : E \to \Sigma_E$ assign vertex and edge labels, and $\tau : E \to \mathbb{R}$ assigns timestamps.

A temporal edge $e \in E$ is written as (u, v) with source $src(e) = u$, destination $dst(e) = v$, edge label $\lambda_E(e)$, and timestamp $\tau(e)$. For a vertex u, let $out(u) = \{e \in E \mid src(e) = u\}$ and $in(u) = \{e \in E \mid dst(e) = u\}$ denote its outgoing and incoming temporal edges, respectively.

We assume that temporal edges are totally ordered. Concretely, edges are sorted by increasing timestamp and ties are broken by an arbitrary but fixed rule (e.g., edge id), yielding a total order

$$e_1 \prec e_2 \prec \cdots \prec e_m, \qquad m = |E|.$$

Thus $e_i \prec e_j$ if $\tau(e_i) < \tau(e_j)$, or $\tau(e_i) = \tau(e_j)$ and $i < j$ after tie breaking.

2.2 Seed-Rooted Temporal Motifs

Given a seed vertex $u \in V$, we consider motifs that are rooted at u via their earliest temporal edge.

Definition 2.2 (Seed-rooted temporal motif instance)**.** Given a seed vertex $u \in V$, a motif size $L \geq 1$, and a time window $\delta > 0$, an ordered sequence of distinct temporal edges $M = (e_1 \prec e_2 \prec \cdots \prec e_L)$ is a *seed-rooted temporal motif instance at u* if it satisfies:

(1) **Anchoring.** The source vertex of the earliest edge is the seed: $src(e_1) = u$.
(2) **δ-window.** The time span is at most δ: $\tau(e_L) - \tau(e_1) \leq \delta$.
(3) **Connectivity.** Ignoring timestamps, the subgraph induced by $\{e_1, \ldots, e_L\}$ is weakly connected (i.e., connected when edge directions are ignored).

Table 1: Summary of key notation.

Symbol	Description
$\mathcal{T} = (V, E, \lambda_V, \lambda_E, \tau)$	Labeled temporal graph
$L,\ \delta$	Motif size (number of edges), time window
$\mathbf{TMV}_u$	Temporal motif vector of seed u
e_0	Anchor edge (earliest selected edge from seed)
$\sigma(e_0)$	Anchor-local chronological suffix
$\mathcal{F}_i(P)$	Live frontier at decision boundary i
ϕ_P	Canonical role assignment
$\rho(P) = (\Lambda(P), \Gamma(P))$	Canonical prefix descriptor
$F_i(P)$	Frontier encoding (live vertices + roles)
$\mathcal{A}_i(P) = (\rho(P), F_i(P))$	ETFA state
$\eta_i(P)$	Next expiration boundary
$R(\mathcal{A}),\ d(\mathcal{A}),\ s(\mathcal{A})$	Remaining budget, connectivity debt, slack

With tie breaking and anchoring, each instance is uniquely represented by its edge sequence and uniquely assigned to its seed.

2.3 Motif Classes and Temporal Motif Vectors

We aggregate motif instances by motif class. Intuitively, two seed-rooted temporal motif instances belong to the same class if they become identical after relabeling vertex IDs while preserving vertex labels, edge labels, directions, and the relative temporal order.

Definition 2.3 (Isomorphic motif instances)**.** Let $M = (e_1, \ldots, e_L)$ and $M' = (e'_1, \ldots, e'_L)$ be two seed-rooted temporal motif instances with vertex sets $V(M)$ and $V(M')$, respectively. We say M and M' are *isomorphic* if there exists a bijection

$$\phi : V(M) \to V(M')$$

such that (i) ϕ maps the seed of M to the seed of M', (ii) $\lambda_V(v) = \lambda_V(\phi(v))$ for all $v \in V(M)$, and (iii) for each $1 \le i \le L$, if $e_i = (x_i, y_i)$ then $e'_i = (\phi(x_i), \phi(y_i))$ and $\lambda_E(e_i) = \lambda_E(e'_i)$. An equivalence class under this relation is called a *motif class*.

Definition 2.4 (Temporal motif vector)**.** Given $L \ge 1$ and $\delta > 0$, let C_L denote the set of motif classes of size L. For a seed vertex u, the *temporal motif vector* of u is the vector $\mathbf{TMV}_u \in \mathbb{N}^{|C_L|}$ indexed by motif classes $C \in C_L$, where

$$(\mathbf{TMV}_u)_C = \left|\{\, M \mid \mathsf{seed}(M) = u,\ \mathsf{class}(M) = C \,\}\right|.$$

2.4 Counting Problem

Problem 1 (Seed-rooted temporal motif counting)**.**

Input. A labeled temporal graph $\mathcal{T} = (V, E, \lambda_V, \lambda_E, \tau)$, a seed vertex $u \in V$, a motif size $L \ge 1$, and a time window $\delta > 0$.

Output. The temporal motif vector $\mathbf{TMV}_u$.

Example 1. For illustration, consider the temporal graph in Figure 2a. Under motif size $L = 3$ and time window $\delta = 50$, the instance in Figure 2b is invalid because $70 - 10 = 60 > \delta$, while Figure 2c shows a valid seed-rooted temporal motif. For this seed, the temporal motif instances fall into the motif classes summarized by the temporal motif vector in Figure 2d.

3 Temporal Future-Equivalent State Compression

Our approach has two main components. This section introduces *temporal future-equivalent state compression*, which collapses many concrete partial histories into one compressed counting state. Section 4 then adds *slack-aware completion pruning*, an orthogonal feasibility filter that discards states that can no longer complete into a valid motif.

Key idea. Existing chronological methods expand concrete partial realizations directly, which is costly because many different histories become indistinguishable once some realized vertices no longer appear in the remaining suffix. FastTGM exploits this by mapping all such *temporally future-equivalent* histories to one compressed state carrying a multiplicity counter, so that exact continuation counting is performed once per equivalence class rather than once per concrete history.

Temporal design rationale. State sharing through frontier-based search [17, 20, 41, 42, 48] is effective in static graph enumeration, but temporal motifs pose a distinct challenge: many realizations may share the same projected graph yet contribute different continuation multiplicities, because the processing order is dictated by motif chronology and the admissible future is anchor-local under the time window (Example 3 illustrates this concretely). Hence the compression must preserve exact counts over the event sequence itself, not merely over the projected graph. This leads to an *Expiring Temporal Frontier Automaton* (ETFA), whose states preserve the canonical motif prefix, the current connectivity pattern, and the concrete vertices that remain visible to the future chronological suffix. To our knowledge, this is the first compressed counting framework for temporal motifs that replaces occurrence-by-occurrence search with equivalence-class-level counting over the chronological suffix.

3.1 Chronological Suffixes and Live Frontiers

To compute $\mathbf{TMV}_u$, FastTGM iterates over every outgoing temporal edge of the seed u as the *anchor edge* e_0, and independently counts the motif instances whose earliest selected edge is e_0. The contributions of all anchors are then summed. Below, we fix one anchor edge and develop the notation for its chronological search space.

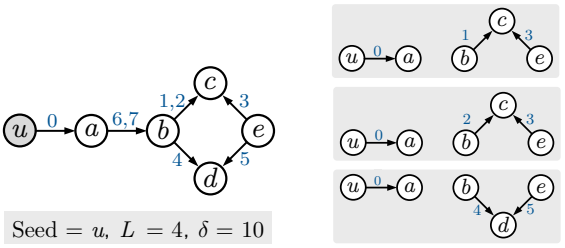
Definition 3.1 (Anchor-local chronological suffix)**.** For a fixed anchor edge e_0, the *anchor-local chronological suffix*

$$\sigma(e_0) = \langle e_1, e_2, \ldots, e_m \rangle$$

is the time-ordered sequence of temporal edges e satisfying 1) $e_0 \prec e$ and $\tau(e) - \tau(e_0) \le \delta$, 2) the endpoints of e are within at most L hops from the seed in the static projection of $\mathcal{T}$ (a necessary condition, since any weakly connected subgraph on L edges has undirected diameter at most L). A *decision boundary* $i \in \{1, \ldots, m+1\}$ means that the first $i-1$ suffix edges have already been decided, and e_i is the next undecided edge.

Intuitively, $\sigma(e_0)$ is the candidate edge set for expanding motifs from anchor e_0; all future relevance below is evaluated only within this suffix. At a decision boundary i, a partial realization has already selected some edges, and only the realized vertices that can still interact with the remaining suffix matter.

Definition 3.2 (Live frontier)**.** Fix a decision boundary i and a partial realization P. For a realized concrete vertex v, let $last_i(v)$ be the last suffix position $j \ge i$ such that e_j is incident to v, if such a position exists. We say that v is *live* at boundary i if $last_i(v)$ exists. The *live frontier* of P at boundary i, denoted by $\mathcal{F}_i(P)$, is the set of all such live realized vertices.



(a) Example graph with an anchor edge.

(b) Concrete prefixes merged by temporal future equivalence.

Figure 3: Running example for ETFA.

The live frontier is the only part of the concrete history that remains directly visible to the future suffix:

- If a realized vertex is still *live*, a future suffix edge may attach to it and change the continuation count.
- If a realized vertex has already *expired*, its concrete identity can no longer be referenced by the remaining suffix.
- After expiration, that vertex survives only through the *canonical motif prefix* already formed.

Example 2 (Live frontier)**.** We illustrate these concepts with the more detailed running example in Figure 3a. The anchor edge is $e_0 = (u, a, 0)$, and the time window is $\delta = 10$. Hence the anchor-local suffix contains every temporal edge after e_0 in the figure. To keep the illustration focused, we count only one fixed seed-rooted target motif of length $L = 4$, namely the final accepted pattern later shown in Figure 4, rather than all motif types induced by the suffix. Consider the top state in Figure 3b. At $t = 4$, its live frontier is $\{a, b, e\}$. At $t = 5$, the live frontier shrinks to $\{a, b\}$ because e has no future incident suffix edge left.

The key consequence is that, once the future suffix is fixed, FastTGM only needs to retain the canonical prefix already selected and the live concrete vertices still visible to future suffix edges.

3.2 ETFA States and Temporal Future Equivalence

To exploit the live-frontier view, FastTGM maps many concrete partial realizations to one *ETFA state*. The construction has two parts:

(1) a *canonical motif-side descriptor* for the realized prefix; and
(2) a *graph-side frontier view* for the concrete vertices that remain live.

The role of an ETFA state is not merely to compress implementation states. It serves as a temporal sufficient statistic for exact continuation counting from a fixed decision boundary in the anchor-local chronological suffix.

Definition 3.3 (Canonical roles)**.** Fix one anchor-local search and a partial realization P. The *canonical role assignment* ϕ_P numbers the concrete vertices appearing in P by their order of first appearance in the selected chronological prefix.

The role assignment removes irrelevant concrete vertex IDs on the *motif side*, while preserving the chronological structure of the realized prefix.

Definition 3.4 (Canonical chronological prefix descriptor)**.** Let P be a partial realization with canonical role assignment ϕ_P. Suppose P realizes roles $\{0, 1, \ldots, q-1\}$ and selected edges $\hat{e}_1 \prec \cdots \prec \hat{e}_k$. We define its *canonical chronological prefix descriptor*

$$\rho(P) = \big(\Lambda(P), \Gamma(P)\big),$$

where:

- $\Lambda(P) = \langle \lambda_0, \lambda_1, \ldots, \lambda_{q-1} \rangle$ is the label sequence of realized canonical roles, with λ_r the label of the concrete vertex assigned to role r;
- $\Gamma(P) = \langle \gamma_1, \gamma_2, \ldots, \gamma_k \rangle$ is the chronological edge-token sequence, and each token
$$\gamma_t = \big(\phi_P(src(\hat{e}_t)), \phi_P(dst(\hat{e}_t)), \lambda_E(\hat{e}_t)\big)$$
records the directed role pair and edge label of the t-th selected edge.

For instance, in Figure 3a, the anchor edge $e_0 = (u, a, 0)$ induces the initial prefix with roles $\phi(u) = 0$ and $\phi(a) = 1$, giving $\Gamma = \langle (0, 1) \rangle$ (labels omitted for clarity as in all running examples).

Frontier encoding. Fix a decision boundary i and a partial realization P. Let $\mathcal{F}_i(P)$ be its live frontier. We encode it as

$$F_i(P) = \langle (v_1, r_1), (v_2, r_2), \ldots, (v_t, r_t) \rangle,$$

where each $v_j \in \mathcal{F}_i(P)$ is a live concrete vertex and $r_j = \phi_P(v_j)$ is its canonical role. The entries are stored in a fixed order on concrete vertices. Since $\rho(P)$ already determines the realized prefix graph over canonical roles, the component membership of each frontier entry can be recovered from $\rho(P)$. Therefore, the frontier view only needs to remember which live concrete vertices remain visible to the future suffix and which canonical roles they instantiate.

Definition 3.5 (ETFA state)**.** Fix a decision boundary i and a concrete partial realization P. Its ETFA state is the pair

$$\mathcal{A}_i(P) = \big(\rho(P), F_i(P)\big).$$

Intuitively, $\rho(P)$ captures the realized motif-side history while $F_i(P)$ records the graph-side vertices still visible to the remaining suffix. ETFA thus forgets expired concrete details while preserving all information needed for continuation counting.

Definition 3.6 (Temporal future equivalence)**.** Two partial realizations P and Q at the same decision boundary i are *temporally future-equivalent* if $\mathcal{A}_i(P) = \mathcal{A}_i(Q)$.

Each ETFA state is stored together with a *state multiplicity*, namely the number of concrete partial realizations mapped to that state at the same boundary. Once two histories are *temporally future-equivalent*, FastTGM merges them and sums their multiplicities.

Example 3 (Temporal future equivalence)**.** Figure 3b zooms into the three concrete prefixes that become temporally future-equivalent at $t = 5$. The upper two prefixes use the concrete vertex c as the internal meeting point, while the lower one uses d. These concrete histories are therefore different before expiration. However, after processing the suffix up to $t = 5$, the internal vertex in each prefix has no future incident suffix edge left. At that point, all three prefixes induce the same canonical chronological prefix descriptor and the same frontier encoding, and thus collapse to one ETFA state. This local view is exactly the same-boundary merge that later reappears as multiplicity growth in Figure 4.

Algorithm 1: ETFA-CountAnchor

Input: seed vertex u, anchor edge e_0, motif size L, time window δ
Output: the contribution of e_0 to $\mathbf{TMV}_u$

$\sigma(e_0) \leftarrow$ the anchor-local chronological suffix of e_0;
Buckets$[1 \dots |\sigma(e_0)| + 1] \leftarrow \emptyset$;
$\mathcal{A}_1 \leftarrow$ `InitAnchorState`(u, e_0);
Buckets$[1]$.`insert`$(\mathcal{A}_1, 1)$;
for $i \leftarrow 1$ **to** $|\sigma(e_0)| + 1$ **do**
  if Buckets$[i] = \emptyset$ **then**
    continue;
  Buckets$[i] \leftarrow$ `ReduceBucket`(Buckets$[i]$);
  foreach $(\mathcal{A}, c) \in$ Buckets$[i]$ **do**
    `ProcessState`$(\mathcal{A}, c, i, \sigma(e_0),$ Buckets$)$;
Procedure `ProcessState`$(\mathcal{A}_i(P), c, i, \sigma(e_0),$ Buckets$)$
  if $\mathcal{A}_i(P)$ *is accepting* **then**
    $\mathbf{TMV}_u[\rho(P)] \leftarrow \mathbf{TMV}_u[\rho(P)] + c$;
    return;
  $\eta \leftarrow$ `NextExpireBoundary`$(\mathcal{A}_i(P))$;
  foreach *take position* $j \in [i, \eta]$ **do**
    $\mathcal{A}' \leftarrow$ `TakeAndExpire`$(\mathcal{A}_i(P), e_j)$;
    if $\mathcal{A}'$ *is feasible* **then**
      Buckets$[j + 1]$.`insert`$(\mathcal{A}', c)$;
  if $\eta \le |\sigma(e_0)|$ **then**
    $\widetilde{\mathcal{A}} \leftarrow$ `ImplicitSkip`$(\mathcal{A}_i(P), \eta)$;
    if $\widetilde{\mathcal{A}}$ *is feasible* **then**
      Buckets$[\eta + 1]$.`insert`$(\widetilde{\mathcal{A}}, c)$;
end

This state view turns anchor-level temporal motif counting into *exact continuation counting over temporal future-equivalence classes*: instead of continuing from each concrete history separately, FastTGM continues from one ETFA state per mergeable class at each decision boundary.

3.3 State Expansion and Counting

This subsection describes the concrete expansion and counting procedure that realizes the temporal future-equivalence view. Counting proceeds boundary by boundary over $\sigma(e_0)$, and all merges happen only among states that reach the *same* decision boundary. Operationally, buckets are only an execution device for grouping states by boundary; the conceptual step is that `ReduceBucket` merges temporally future-equivalent states before further expansion. Algorithm 1 summarizes this anchor-level routine. At a high level, each nonterminal state at boundary i either *takes* a suffix edge e_j (adding it to the realization and advancing to boundary $j + 1$), or *implicitly skips* all unchosen edges up to the next expiration point. After every advance, *expiration* removes frontier vertices whose last future incidence has passed. We detail these operations below.

Expansion rules. We next unpack the three boundary-local operations used by `ProcessState`. Among them, *take* and *implicit skip* produce successor states, while *expiration* is the deterministic state update that is applied when the boundary advances. Fix a feasible nonterminal state $\mathcal{A}_i(P) = (\rho(P), F_i(P))$.

Take. A take transition selects one suffix edge $e_j = (u, v)$ with $j \ge i$ and forms $P' = P \cup \{e_j\}$. On the motif side, the transition either reuses existing canonical roles, introduces one new role, or creates a new realized component. On the graph side, it updates the current component partition and refreshes the live frontier after position j. A successor is discarded immediately if it violates the remaining budget, loses necessary future-visible support, or can no longer be completed into a connected motif.

Expiration. After the decision at boundary j, every live vertex whose last future incidence is j expires and is removed from the frontier. This is the point where ETFA safely forgets concrete identities that are no longer visible to the remaining suffix. If expiration leaves an unfinished component without any live support, the successor becomes infeasible.

Next expiration boundary. For every feasible nonterminal state, the live frontier is nonempty. We therefore define

$$\eta_i(P) = \min_{v \in \mathcal{F}_i(P)} last_i(v),$$

namely the earliest suffix position at which one currently live frontier vertex must expire.

Implicit skip. Between the current boundary i and $\eta_i(P)$, repeatedly excluding suffix edges does not change $\rho(P)$, the realized connectivity pattern, or the current live frontier. Therefore, FastTGM replaces the entire exclude-only run with one *implicit-skip* successor to boundary $\eta_i(P) + 1$, with expiration applied once. The following lemma formalizes the exactness of this compression.

Lemma 1 (Exactness of implicit skip)**.** Let $\mathcal{A}_i(P)$ be a feasible nonterminal ETFA state, and let $\eta_i(P)$ be its next expiration boundary. Along any exclude-only path from boundary i through position $\eta_i(P)$, the selected partial realization remains P, no live frontier vertex expires before $\eta_i(P)$, and the first state change occurs only when expiration is applied at $\eta_i(P)$. Consequently, the entire exclude-only interval can be replaced by one implicit-skip transition to boundary $\eta_i(P) + 1$ without affecting the resulting counts.

By Lemma 1, each state only needs to consider feasible takes inside the current active interval and one skip successor induced by the next expiration boundary.

Example 4 (State expansion and multiplicity)**.** Figure 4 traces the anchor-level execution for the fixed target motif. Each dashed rectangle denotes one ETFA state, and the orange-highlighted vertices inside a state form its current temporal live frontier; a dashed circle marks an expiring vertex. The label $\times k$ gives the state multiplicity, and each red-cross marker denotes an omitted branch that cannot realize the target motif. For example, at $t = 5$ the two upper surviving branches have already merged into one state with multiplicity 3, while the lower branch remains a separate state with the same canonical prefix but a different live frontier. Later, same-boundary reductions continue to aggregate equivalent histories, and by $t = 7$ all surviving branches collapse to one accepting state with multiplicity 6. Thus, instead of expanding six separate concrete continuations at the end of the search, ETFA counts them once through one compressed accepting state.

Reduction at a boundary. Before any further expansion at boundary i, `ReduceBucket` merges states that share the same pair (ρ, F) and sums their multiplicities. States from different boundaries are

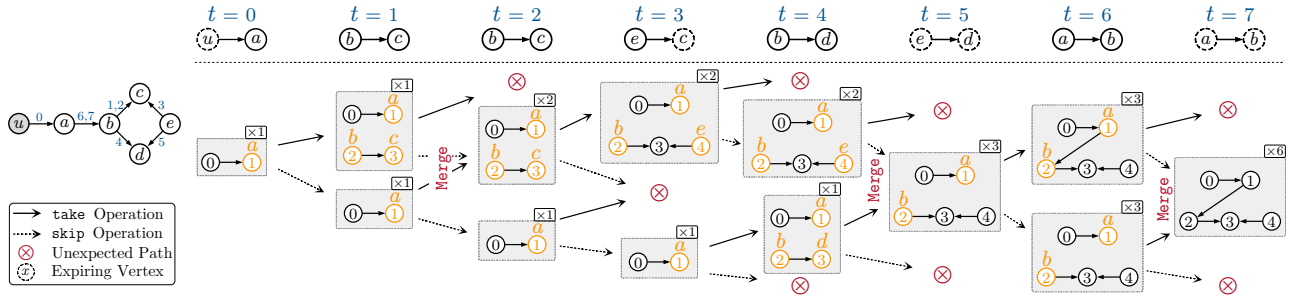


Figure 4: Chronological execution trace and temporal future-equivalent state compression.

never merged. This *same-boundary reduction* is simply the operational realization of temporal future equivalence under the fixed remaining suffix. When a reduced state is accepting, its multiplicity is added directly to $\mathbf{TMV}_u$ rather than expanded further.

3.4 Correctness

Proofs of Lemmas 1 and 3 are provided in Appendix A.

Proof strategy. Fix an anchor edge e_0 and its suffix $\sigma(e_0)$. For a partial realization P at boundary i, let $\mathsf{TakeSucc}(P)$ and $\mathsf{SkipSucc}(P)$ denote the multisets of feasible take and implicit-skip successors produced by `ProcessState`, and define

$$\mathsf{Succ}(P) = \mathsf{TakeSucc}(P) \uplus \mathsf{SkipSucc}(P).$$

Let $\mathsf{Cont}(P)$ be the multiset of accepting canonical descriptors reachable from P, defined recursively by

$$\mathsf{Cont}(P) = \begin{cases} \{\{\rho(P)\}\}, & \text{if } P \text{ is accepting,} \\ \biguplus_{P' \in \mathsf{Succ}(P)} \mathsf{Cont}(P'), & \text{otherwise.} \end{cases}$$

Correctness follows from three facts:

(1) temporally future-equivalent histories have identical continuation multisets $\mathsf{Cont}(\cdot)$;
(2) implicit skip is exact by Lemma 1;
(3) same-boundary reduction preserves counts under multiplicity summation.

Lemma 2 (Temporal state sufficiency)**.** Let P and Q be partial realizations at the same boundary i. If $\mathcal{A}_i(P) = \mathcal{A}_i(Q)$, then $\mathsf{Cont}(P) = \mathsf{Cont}(Q)$.

Proof. We induct on the number of remaining suffix positions after boundary i. *Base case.* If P is accepting, then $\mathsf{Cont}(P) = \{\{\rho(P)\}\} = \{\{\rho(Q)\}\} = \mathsf{Cont}(Q)$, since $\rho(P) = \rho(Q)$ by hypothesis. *Inductive step.* The hypothesis implies

$$\rho(P) = \rho(Q), \qquad F_i(P) = F_i(Q), \qquad \eta_i(P) = \eta_i(Q).$$

Hence the two histories expose exactly the same future-visible concrete vertices under the same canonical roles, and they share the same active interval $[i, \eta_i(P)]$. For every take position j in this interval, let P'_j and Q'_j be the corresponding feasible take successors. Then $\mathcal{A}(P'_j) = \mathcal{A}(Q'_j)$, because the motif-side update, expiration behavior, and frontier update are fully determined by (ρ, F_i) together with the chosen suffix edge. The same argument gives identical implicit-skip successors as well. Therefore, $\mathsf{Succ}(P) = \mathsf{Succ}(Q)$. Applying induction on the remaining suffix length yields

$$\mathsf{Cont}(P) = \biguplus_{P' \in \mathsf{Succ}(P)} \mathsf{Cont}(P') = \biguplus_{Q' \in \mathsf{Succ}(Q)} \mathsf{Cont}(Q') = \mathsf{Cont}(Q). \qquad \square$$

Lemma 3 (Correctness of same-boundary reduction)**.** At a fixed boundary i, replacing all bucket entries with the same ETFA state by one entry whose multiplicity is the sum of their multiplicities preserves the final count vector contributed by that boundary.

Theorem 1 (Correctness of ETFA-CountAnchor)**.** For a fixed seed vertex u and anchor edge e_0, Algorithm 1 returns exactly the contribution of all connected length-L temporal motif instances rooted at u whose first selected edge is e_0.

Proof. Let P_0 be the initial anchor realization. By Lemma 1, implicit skips preserve $\mathsf{Cont}(P_0)$. By Lemmas 2 and 3, same-boundary merges preserve $\mathsf{Cont}(P_0)$ under multiplicity summation. Hence for every descriptor ξ,

$$\mathbf{TMV}_u^{(e_0)}[\xi] = \mu_{\mathsf{Cont}(P_0)}(\xi),$$

where $\mu_{\mathsf{Cont}(P_0)}(\xi)$ is the multiplicity of ξ in $\mathsf{Cont}(P_0)$. $\square$

Anchor-to-seed aggregation. Running Algorithm 1 over all anchor edges of the seed vertex and summing their contributions therefore produces the seed-rooted temporal motif vector $\mathbf{TMV}_u$.

3.5 Complexity Analyses

Compressed-state measure. The complexity of ETFA is best expressed in terms of compressed boundary activity rather than raw concrete histories, because the gain comes precisely from collapsing temporal future-equivalence classes. For a fixed anchor edge e_0, let $m = |\sigma(e_0)|$. For each boundary $i \in \{1, \ldots, m+1\}$, let:

- B_i be the number of bucket entries before `ReduceBucket`;
- N_i be the number of distinct ETFA states after reduction;
- T_i be the number of feasible take successors examined from the reduced bucket.

We also let W denote the maximum number of bucket entries stored simultaneously during the processing of this anchor.

Lemma 4 (State representation size)**.** For motif length L, every ETFA state can be stored and compared in $O(L)$ words.

Proof. Its canonical descriptor contains at most L chronological edge tokens. Moreover, each selected edge introduces at most two new canonical roles, so the number of realized roles is at most

$2L$. The ordered live frontier contains at most one entry per realized role and therefore also has size $O(L)$. Hence both storage and comparison cost are $O(L)$. □

Theorem 2 (Per-anchor time and space bounds)**.** For a fixed anchor edge e_0, Algorithm 1 can be implemented in

$$O\left(m + L\sum_{i=1}^{m+1}\left(B_i \log(B_i+1) + N_i + T_i\right)\right)$$

time when `ReduceBucket` uses comparison-based sorting, and in $O(L \cdot W)$ space.

Proof. Building the anchor-local suffix costs $O(m)$. By Lemma 4, each bucket entry has size $O(L)$. Reducing bucket i by sorting and merging equal states therefore costs $O(L \cdot B_i \log(B_i+1))$. After reduction, each of the N_i distinct states performs a constant number of local updates, including the expiration check and at most one implicit-skip emission, for total cost $O(L \cdot N_i)$. The T_i feasible take successors examined from this bucket contribute another $O(L \cdot T_i)$. Summing over all boundaries gives the claimed running time.

For space, each stored bucket entry occupies $O(L)$ words by Lemma 4. If at most W entries are stored simultaneously across all pending buckets, the total working memory is $O(L \cdot W)$. □

Worst-case behavior. The bound above is intentionally stated in terms of compressed boundary activity. In the worst case, no two concrete partial realizations become temporally future-equivalent: each prefix either has a distinct canonical descriptor or exposes a distinct live frontier to the remaining suffix. Then ETFA degenerates to explicit chronological exploration, with only the overhead of maintaining and reducing state keys.

For a fixed anchor edge e_0, let $m = |\sigma(e_0)|$ and define $H_L(m) = \sum_{r=0}^{L-1}\binom{m}{r}$, the number of chronological suffix-prefixes obtained by selecting at most $L-1$ additional edges after the anchor. The worst-case number of distinguishable partial histories is $O(H_L(m))$, which is $O(m^{L-1})$ for fixed L and exponential when L grows with m. Accordingly, the worst-case space is $O(L \cdot H_L(m))$, and the worst-case time is $O(L \cdot H_L(m) \cdot \log H_L(m))$, the logarithmic factor arising from comparison-based bucket reduction.

This worst case is not where ETFA is designed to gain. Its advantage appears when many concrete histories share the same temporal future, i.e., when $N_i \ll B_i$ after same-boundary reduction. In practice, same-boundary reduction sharply decreases N_i relative to B_i; our ablation (Section 6.4) shows a 77–100% reduction in the peak number of states at any decision boundary.

4 Slack-Aware Completion Pruning

ETFA removes redundancy among temporally future-equivalent counting states. A separate issue is *completion feasibility*: even after a take transition produces a locally valid child state, that state may already be impossible to complete into a connected seed-rooted temporal motif within the remaining chronological suffix. FastTGM addresses this issue with *slack-aware completion pruning*. The key observation is that each take transition consumes a small amount of future *non-bridge budget*, so the residual slack of the child state directly measures how much structural freedom remains in the suffix. When the slack becomes small, the future structure is tightly constrained, and FastTGM triggers progressively stronger temporal feasibility checks.

4.1 Budget, Connectivity Debt, and Temporal Completion

Definition 4.1 (Remaining budget, connectivity debt, and slack)**.** Let $\mathcal{A} = (\rho, F)$ be a feasible nonterminal counting state. We let $\Pi(\rho)$ denote the connected components of the prefix graph encoded by ρ, and write $\ell(\mathcal{A}) = |\Gamma(\rho)|$, $\kappa(\mathcal{A}) = |\Pi(\rho)|$. Its *remaining budget*, *connectivity debt*, and *slack* are defined as

$$R(\mathcal{A}) = L - \ell(\mathcal{A}), \quad d(\mathcal{A}) = \kappa(\mathcal{A}) - 1, \quad s(\mathcal{A}) = R(\mathcal{A}) - d(\mathcal{A}).$$

The component containing the seed is the *root component*; all other components are *non-root components*.

Thus, $R(\mathcal{A})$ is the number of future selected edges still needed to reach motif length L, $d(\mathcal{A})$ is the minimum number of future component-reducing edges still needed to reconnect the current realization, and $s(\mathcal{A})$ is the residual budget for future selected edges that *do not* immediately pay this connectivity debt.

Temporal completion feasibility. This completion question is not captured by static connectivity alone. Because ETFA already removes states whose disconnected pieces lose all live support after expiration, every non-root component still counted by $d(\mathcal{A})$ remains, in principle, reconnectable. However, reconnectability now depends on the *remaining chronological suffix*: a bridge in the static projection is useful only if it is still realizable in time and before the frontier support needed for that reconnection disappears. Thus the relevant object is not generic residual connectivity, but *temporal completion feasibility* under the remaining suffix and the current live frontier.

Transition-cost view. Consider a take-generated child state $\mathcal{A}'$ obtained from $\mathcal{A}$. A selected edge is of one of three types: *bridge*, *attach / same-component*, or *spawn*. These three cases change slack in an exact and simple way: (1) bridge: $s(\mathcal{A}') = s(\mathcal{A})$; (2) attach/same-component: $s(\mathcal{A}') = s(\mathcal{A}) - 1$; and (3) spawn: $s(\mathcal{A}') = s(\mathcal{A}) - 2$. A bridge consumes one unit of remaining edge budget and one unit of connectivity debt, so the slack is unchanged. An attach step consumes one future edge without reducing the component count, so the slack drops by one. A spawn step both consumes one future edge and increases the future connectivity debt by one, so the slack drops by two. Hence ETFA's pruning policy can be viewed as a residual-budget rule: a child state is promising only if its remaining slack is still large enough to support the future non-bridge work required for completion.

Remark 1. Slack also admits the closed form $s(\mathcal{A}) = L - 1 - n_{\text{att}}(\mathcal{A}) - 2n_{\text{sp}}(\mathcal{A})$, where n_{att} and n_{sp} count the post-anchor attach and spawn edges, respectively. Hence bridge-heavy prefixes retain large slack late into the search, while attach- or spawn-heavy prefixes enter the constrained regime earlier.

4.2 Low-Slack Temporal Certificates

The residual-budget view suggests a general principle: a take-generated child should be kept only if its remaining slack can still cover the minimum future non-bridge cost required for completion. Computing that minimum exactly for an arbitrary residual suffix is too expensive to use as an inner-loop test. The useful regime is

Algorithm 2: CompletionPrune

Input: low-slack child state $\mathcal{A}$ at boundary i
Output: KEEP or PRUNE

```
if s(A) = 0 then
    restrict future admissible takes to bridge-type suffix edges;
    if DirectReconnectable(A, i) = FALSE then
        return PRUNE;
else if s(A) = 1 then
    discard spawn-type suffix edges from future admissible takes;
    if TwoStepReconnectable(A, i) = FALSE then
        return PRUNE;
if FallbackReachable(A, i, R(A)) = FALSE then
    return PRUNE;
return KEEP;
```

therefore the one where the budget itself becomes tight enough to force a very restricted future structure.

Low-slack implications. The transition-cost view makes the constrained regimes explicit. If $s(\mathcal{A}) = 0$, then every future selected edge must be a bridge, so each non-root component must still admit a direct future reconnection to another current live component. If $s(\mathcal{A}) = 1$, then the remaining suffix may contain at most one attach / same-component step and no spawn step, so each non-root component must still be reconnectable within at most two future selected edges. These low-slack conditions turn temporal completion into a short-horizon suffix-local question, which is exactly where efficient temporal certificates become possible.

Transition-level filtering. Before materializing a child state, Algorithm 2 already filters inadmissible take types: at $s = 0$ only bridge edges remain admissible; at $s = 1$ spawn edges are excluded.

Soundness and incompleteness. In the implementation, these low-slack certificates are checked by batched constant-depth scans over per-vertex future suffix-edge lists induced by the current live frontier. The routine reuses these future edge lists to test the required reconnection opportunities for all current target components simultaneously, and does not materialize an auxiliary graph. The resulting tests are necessary but not sufficient: different components may each look locally reconnectable in isolation while the same suffix still cannot realize all of them together. This is why FastTGM adds a stronger bounded fallback after the local low-slack certificates.

4.3 The CompletionPrune Routine

Algorithm 2 attaches completion pruning to low-slack take expansions. It shows the child-level pruning routine applied after the suffix-level filtering described above.

Bounded fallback. `FallbackReachable` handles the ambiguous survivors of the local certificates via a bounded BFS over an implicit temporal reachability space. Its nodes are the current live connected components of the counting state and future concrete vertices still exposable by suffix edges. The graph is not materialized: adjacency is generated on demand from the same per-vertex future edge lists used by the local certificates. From a live component, the search follows future suffix edges incident to its live frontier vertices; from a future concrete vertex, it follows that vertex's own future suffix edges in the remaining suffix. Each BFS depth corresponds to one future selected edge, and the depth bound is exactly $R(\mathcal{A})$. If some non-root component cannot be reached from the root side within this budget, the state is pruned.

Selective activation. The fallback is invoked only when $s(\mathcal{A}) \le 2$. This threshold is an efficiency choice: for larger slack, the local certificates become much weaker while the bounded search becomes less selective and more expensive. Under low slack, non-bridge freedom is already scarce, so the bounded search remains informative.

Timing of low slack. The low-slack regime cannot appear arbitrarily early. Since every current component must contain at least one selected edge, we have $\kappa(\mathcal{A}) \le \ell(\mathcal{A})$. Therefore, $s(\mathcal{A}) = L - \ell(\mathcal{A}) - \kappa(\mathcal{A}) + 1 \ge L - 2\ell(\mathcal{A}) + 1$. Consequently,

$$s(\mathcal{A}) \le 1 \Rightarrow \ell(\mathcal{A}) \ge \left\lceil \frac{L}{2} \right\rceil.$$

That is, states with $s(\mathcal{A}) \le 1$ can emerge only after roughly half the motif edges have been fixed, making low-slack certification a targeted late-stage filter rather than an always-on inner-loop burden.

Lemma 5 (Soundness of completion pruning)**.** If CompletionPrune returns PRUNE for a child state $\mathcal{A}$ at boundary i, then $\mathcal{A}$ admits no feasible completion into a connected seed-rooted temporal motif of length L within the remaining suffix.

Why pruning still helps late in the search. Even late pruning is worthwhile for counting: pruning one compressed state eliminates an entire equivalence class of concrete continuations, so the pruning gain is amplified by ETFA's compression ratio. A single pruned state may already carry high multiplicity and still face a long suffix, making each successful prune far more effective than removing a single concrete branch.

Cost analyses. The completion pruning pipeline has three cost levels. Transition-level filtering is $O(1)$ once the take type of a suffix edge is known. For the local certificates, let $I_{\mathrm{loc}}(\mathcal{A}, i)$ denote the number of future suffix-edge list entries examined by the bounded scan. Then `DirectReconnectable` and `TwoStepReconnectable` each run in $O(I_{\mathrm{loc}}(\mathcal{A}, i))$: they inspect only constant-depth temporal neighborhoods, reuse the same anchor-local per-vertex future edge lists as the main counting routine, and avoid constructing any auxiliary graph. The bounded fallback is linear in the explored temporal-reachability states and touched future-edge adjacencies within depth $R(\mathcal{A})$, with linear space in the BFS frontier. Since this last layer is invoked only after low-slack filtering and only on ambiguous survivors of the local certificates, its practical cost is concentrated on a much smaller subset of the search space than the full take expansion.

5 Industrial Case Study: Risk Pattern Mining with Temporal Motifs

This section illustrates how FastTGM can support risk-oriented pattern discovery and evaluation in practice. Our goal here is not to introduce another mining algorithm, but to show how efficient seed-rooted temporal motif counting can serve as the feature extraction engine in an industrial workflow.

5.1 Workflow Overview

Figure 5 summarizes the deployment setting. The raw input is a heterogeneous temporal event stream, such as transactions, social interactions, and login events. For a seed entity of interest, the system first constructs a seed-rooted temporal graph inside a chosen

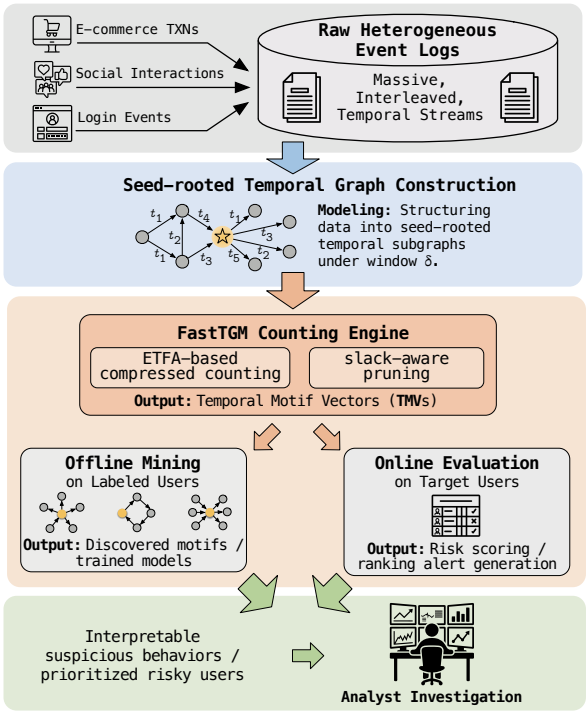


Figure 5: Industrial workflow enabled by FastTGM.

time window, and then invokes FastTGM to count seed-rooted temporal motifs. The resulting TMVs are passed to downstream modules for offline pattern mining and online evaluation.

We use an evolving-graph view with snapshots $\{G_t\}_{t\geq 1}$. The mining stage operates on a snapshot G_T with labeled seeds, while the evaluation stage applies the mined patterns or models to target seeds in a later snapshot G_{T+1}. This separation matches practical deployment: motif extraction is shared across both stages, whereas the downstream decision rule can vary across tasks.

Temporal motif counting dominates this workflow: once TMV extraction becomes tractable, the remaining mining and evaluation stages are lightweight by comparison.

5.2 Industrial Integration

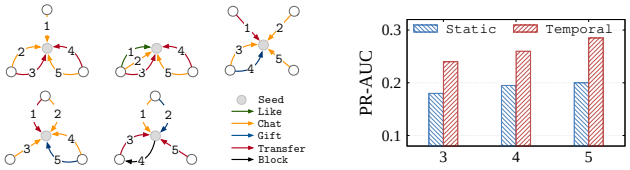
We instantiate the workflow with a simple enumerate–mine–evaluate pipeline. Given labeled seeds in G_T, we first invoke FastTGM to compute their TMVs. Let $\mathcal{S}^+$ and $\mathcal{S}^-$ denote the sets of risky and benign seeds, respectively. For a motif m, let c_m^+ and c_m^- be its total counts over $\mathcal{S}^+$ and $\mathcal{S}^-$, and let n_m^+ be the number of risky seeds whose TMV contains at least one occurrence of m. Following the risk-pattern ranking in GraphRPM [46], we score motifs by

$$\text{Precision}(m) = \frac{c_m^+}{c_m^+ + c_m^-}, \qquad \text{Recall}(m) = \frac{n_m^+}{|\mathcal{S}^+|}, \tag{1}$$

and combine the two via

$$F_\beta(m) = \frac{(1+\beta^2)\,\text{Precision}(m)\,\text{Recall}(m)}{\beta^2\,\text{Precision}(m) + \text{Recall}(m)}. \tag{2}$$

The top-K motifs under F_β are then retained as candidate risk patterns. This ranking-based instantiation is useful when the goal is to surface interpretable temporal risk patterns. The same TMV representation can also support lightweight rule filtering, for example by selecting motifs with high risky-to-benign contrast or motifs that are nearly exclusive to risky seeds.



(a) Temporal risk patterns. (b) PR-AUC by motif size (L).

Figure 6: Industrial case study on `Commerce`.

When the downstream task is prediction rather than pattern discovery, the same TMV features can be fed directly into standard supervised models such as logistic regression, random forests, or XGBoost. Let $h(\mathbf{TMV}_u)$ denote the risk score predicted for seed u by such a model. This model-based variant uses exactly the same temporal motif features extracted by FastTGM, and therefore fits naturally into the same enumerate–mine–evaluate workflow.

For evaluation on G_{T+1}, we again compute the TMV of each target seed u and aggregate the mined motifs into a risk score. A simple pattern-based scoring rule is

$$\text{score}(u) = \sum_{m\in\text{top-}K} \mathbf{1}[(\mathbf{TMV}_u)_m > 0] \cdot F_\beta(m), \tag{3}$$

where $\mathbf{1}[\cdot]$ indicates whether u contains at least one instance of motif m. This score can be used directly for ranking, or thresholded to trigger alarms. In the model-based variant, the evaluation score is simply $h(\mathbf{TMV}_u)$, which can again be used for ranking or thresholded for alarm generation. The key point is unchanged: the downstream module always consumes the same TMV representation produced by FastTGM, keeping feature extraction consistent between mining and deployment.

5.3 Case Study Findings

We instantiate the above workflow on `Commerce`, an industrial commerce interaction graph from our industry partner ABC, covering 1.2B temporal events over one quarter. It contains multi-relational events such as social interactions and money-transfer behaviors. We mine risk patterns on a July 2025 snapshot and evaluate them on a disjoint weekly snapshot from September 2025, following the partner's production workflow. Due to privacy constraints, we report the graph scale and temporal split but omit partner-specific thresholds and detailed configuration of the downstream mining module.

Discovered risk patterns. Figure 6a shows representative temporal risk motifs found by the workflow. They reveal a recurring behavioral signature in which a risky account interacts with multiple victims through tightly coordinated actions, for example by messaging, receiving transfers or gifts, and then blocking the victims. Such patterns are difficult to capture with static motifs because both the event order and the short temporal span are essential.

Predictive utility and early detection. Figure 6b compares static and temporal motif features on the September weekly snapshot, where fraudulent users account for only about 1% of the week's new users. PR-AUC, i.e., the area under the precision-recall curve, is a

Table 2: Dataset statistics (1K=10^3, 1M=10^6, 1B=10^9).

Dataset	$n = \|V\|$	$\|E\|$	$\|E_T\|$	Time Span
SuperUser	192.4K	892.1K	1.1M	7.6 years
WikiTalk	1.1M	3.1M	6.1M	6.2 years
StackOverflow	6.0M	32.7M	41.5M	7.6 years
Bitcoin	48.1M	86.8M	113.1M	7.1 years
Reddit	8.4M	517.2M	636.3M	10.1 years
Commerce	23.4M	41.0M	1.2B	1.0 quarter

standard metric for such extreme-imbalance settings. Temporal motif features consistently outperform static motif features, achieving PR-AUC of 0.24–0.285 versus 0.18–0.20 for static features (33%–43% higher), with the margin widening from $L = 3$ to 5. This indicates not only that temporal ordering contributes substantial discriminative signal beyond static structure, but also that supporting larger motif sizes is practically important because the additional temporal context improves ranking quality. During that week, domain experts newly labeled 185 users as risky based on manual investigation of the results produced by the existing static-motif workflow. By applying the temporal risk patterns mined from July, FastTGM identified 34 additional users who were not flagged in September. These 34 users were later confirmed as fraudulent in October, one month after the evaluation snapshot, suggesting that temporal motif signals can detect emerging risks ahead of manual review.

6 Experiments

We evaluate FastTGM for seed-rooted temporal motif counting against strong baselines on public and industrial temporal graphs.

6.1 Experimental Setup

Environment. All experiments are conducted on a server with two Intel Xeon(R) Platinum 8336C CPUs @ 2.30GHz (64 physical cores) and 2 TB RAM, running Debian 9.13. Hyper-threading is disabled.

Datasets. We evaluate on five public temporal graphs and one industrial dataset. Table 2 summarizes their statistics. SuperUser, WikiTalk, and StackOverflow are obtained from SNAP [26]. SuperUser and StackOverflow are user-interaction graphs from Stack Exchange forums, where a temporal edge records a reply or comment event. WikiTalk captures timestamped interactions on Wikipedia talk pages. Bitcoin is a Bitcoin transaction network [22], and Reddit is a temporal reply graph derived from Reddit comments [14]. Commerce is an industrial commerce interaction graph collected from ABC (the counterpart of TikTok).

Comparisons and implementation. We compare FastTGM with five baselines. Since no published method directly targets seed-rooted all-pattern temporal motif counting, we implement three baselines that cover the main design paradigms (eDFS, EComb, EComb+). We further include two published query-based temporal matching methods (TemporalRI, Mackey) adapted to our setting. Detailed pseudocode for the baselines is provided in Appendix B.

- eDFS: an incidence-driven DFS baseline that expands motifs by following currently incident temporal edges. Since the same motif can be reached through multiple expansion orders, eDFS performs deduplication by canonical hashing of completed occurrences.
- EComb: a chronological combinational baseline that enumerates anchor-local edge subsets in timestamp order and performs connectivity checking after selection.
- EComb+: extends EComb with budget-based connectivity pruning that filters candidate edges by their structural effect (bridge, attach, or spawn) on the current partial realization, but still enumerates each concrete realization individually.
- TemporalRI [33]: an RI-style temporal subgraph isomorphism engine that combines query-guided candidate refinement with temporal-consistency checks for multiple contacts.
- Mackey [32]: a chronological temporal matching engine that searches occurrences under a time-respecting edge order and is optimized for query-oriented matching workloads.

All compared methods are integrated into the same C++20 framework and compiled with -O3. For TemporalRI and Mackey, we first enumerate the canonical connected seed-rooted temporal motif patterns of size L, then issue one query per pattern, and finally aggregate the returned counts and runtimes into the seed-rooted temporal motif vector result.

Parameter settings. For each dataset, we sample 1,000 seed vertices uniformly at random and report average per-seed results unless stated otherwise. We test motif sizes $L \in \{3, 4, 5, 6\}$ and use $L = 5$ by default. The default time window δ is set according to the temporal edge intensity $|E_T|/time_span$ to obtain comparable local event densities across datasets: 1 day for SuperUser, WikiTalk, and StackOverflow, and 1 hour for Bitcoin, Reddit, and Commerce.

6.2 Overall Performance

Table 3 reports the average runtime per seed (seconds) over 1,000 sampled seeds. Gray cells indicate the 3-hour timeout.

Small motifs ($L = 3, 4$) leave less room for compression. For smaller motifs, the search depth is still shallow, and the strongest baseline is often EComb+ rather than FastTGM. At $L = 3$, EComb+ is the fastest method on all six datasets. At $L = 4$, FastTGM becomes the fastest on SuperUser, WikiTalk, and Commerce, while EComb+ remains best on StackOverflow, Bitcoin, and Reddit. This behavior is expected: when the suffix space is still small, the number of distinct concrete histories is already modest, so there are few future-equivalent states to merge. The per-state bookkeeping of FastTGM (canonical descriptors, frontier encoding, bucket sorting) is not yet amortized by sufficient compression gain. Even at these sizes, the query-based baselines TemporalRI and Mackey are already noticeably slower on most datasets because they execute the workload as many pattern-specific queries rather than one shared counting pass.

At $L = 5$, compression starts to pay off. Once the motif size reaches $L = 5$, FastTGM becomes the fastest method on five of the six datasets: SuperUser (0.47 s), WikiTalk (2.36 s), StackOverflow (2.11 s), Bitcoin (7.01 s), and Commerce (2.98 s). The gaps are already substantial on several graphs: FastTGM is 4.2× faster than EComb+ on SuperUser, 68× faster on WikiTalk, 3.6× faster on StackOverflow, and about 15.6× faster on Commerce. Reddit is the main exception, where EComb+ still gives the best time (11.94 s versus 34.27 s for FastTGM). Overall, $L = 5$ marks the point where future-equivalent state compression and completion pruning begin to pay for their bookkeeping cost.

Table 3: Overall performance comparison (seconds). Gray cells denote 3-hour timeouts (TLE).

Datasets	eDFS	EComb	EComb+	TemporalRI	Mackey	FastTGM	eDFS	EComb	EComb+	TemporalRI	Mackey	FastTGM
	$L = 3$						$L = 4$					
SuperUser	0.03	0.03	**0.02**	2.83	0.59	0.03	0.64	0.96	0.06	14.35	14.44	**0.05**
WikiTalk	0.09	0.10	**0.01**	0.39	0.13	0.09	11.74	24.70	0.49	2.90	3.46	**0.18**
StackOverflow	0.83	1.61	**0.02**	2.06	1.33	0.55	1.74	3.76	**0.05**	5.15	3.86	1.84
Bitcoin	3.50	1.77	**0.01**	1.94	1.53	2.94	7.36	6.56	**0.05**	2.05	1.90	4.05
Reddit	8.46	10.79	**0.01**	20.35	18.02	26.25	15.37	14.30	**0.16**	19.38	18.57	21.87
Commerce	21.11	11.19	**0.13**	223.83	203.81	4.44	31.40	36.58	2.38	202.18	188.89	**1.93**
Datasets	$L = 5$						$L = 6$					
SuperUser	80.18	45.16	1.96	174.36	437.84	**0.47**		1370.02	52.39			**9.90**
WikiTalk	5132.14	5891.70	160.94	39.41	707.37	**2.36**				1365.27		**134.50**
StackOverflow	916.10	1142.07	7.61	59.68	117.89	**2.11**			1311.37	2132.87	TLE	**49.23**
Bitcoin	779.73	427.65	8.31	7.81	38.65	**7.01**	TLE		809.68	130.38		**28.99**
Reddit	736.54	476.39	**11.94**	40.54	195.55	34.27			2496.77	943.00		**63.44**
Commerce	2639.99	70.56	46.55	322.12	714.51	**2.98**		1284.40	911.87	1809.69		**7.37**

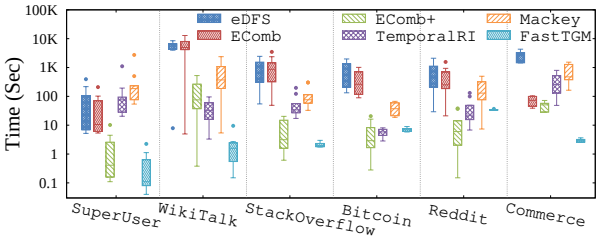


Figure 7: Runtime distribution over sampled seeds.

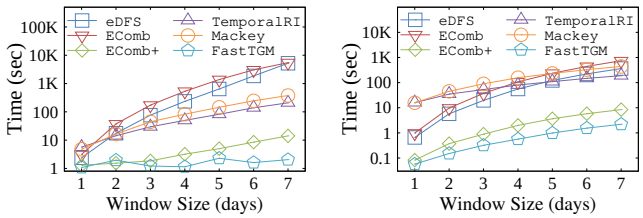


Figure 8: Runtime as the window size varies ($L = 4$).

$L = 6$ exposes the scalability gap. For $L = 6$, FastTGM is the only method that completes on all six datasets within the time budget, and it is the fastest method that completes in every case. The advantage is especially clear on larger graphs: FastTGM reduces runtime from 1311/2133 s (EComb+/TemporalRI) to 49.23 s on StackOverflow, from 943 s (TemporalRI) to 63.44 s on Reddit, and from 912/1810 s (EComb+/TemporalRI) to 7.37 s on Commerce. This is the main takeaway from the table: as motif size grows, the baseline methods either time out or degrade by one to two orders of magnitude, whereas FastTGM remains robust enough to keep all datasets tractable.

6.3 Runtime Characteristics

We next examine two complementary runtime views: per-seed variance and sensitivity to the time-window size.

Runtime distribution across seeds. Figure 7 reports the per-seed runtime distribution under the default setting ($L = 5$). The main pattern is heavy-tailed seed difficulty: TemporalRI and especially Mackey show wide spreads and long upper tails on datasets such as SuperUser, WikiTalk, and Commerce, indicating that a small fraction of hard seeds dominates their runtime. In contrast, FastTGM is markedly more stable on most datasets, with much tighter boxes and shorter upper tails on StackOverflow, Reddit, and Commerce.

On Bitcoin, TemporalRI is competitive in this sampled-seed view, which is consistent with Bitcoin already being one of the easiest datasets for query-based matching under the default 1-hour window. On Reddit, TemporalRI has a lower median on the sampled seeds, but its upper tail is still substantially heavier, whereas FastTGM remains concentrated in a narrow runtime band. Overall, the boxplots show that the benefit of FastTGM is not only lower average time, but also better runtime stability across seeds.

Impact of window size. Figure 8 reports the varying-window experiment, where the window increases from 1 to 7 days under $L = 4$ on StackOverflow and SuperUser. Larger windows admit more temporal edges into the suffix and therefore enlarge the completion search space, so all methods slow down. The sensitivity gap, however, is clear. On SuperUser, FastTGM rises from 0.057 s to 2.205 s, whereas TemporalRI and Mackey increase from 15.9/15.7 s to 188/438 s. On StackOverflow, FastTGM stays in a narrow 1–2 s band across all seven windows, while TemporalRI and Mackey grow to 208 s and 377 s at 7 days. The FastTGM curve on StackOverflow is not perfectly monotone, but the fluctuation is small and does not change the overall trend. This is expected: enlarging the window increases the candidate suffix, but it can also create additional merge and prune opportunities for some seeds. Consequently, the average per-seed runtime need not increase strictly monotonically. Overall, FastTGM is substantially less sensitive to window growth than the query-based baselines.

6.4 Ablation Study

We study four variants of FastTGM: FastTGM-Plain disables both merge and prune, FastTGM-Merge enables only merge, FastTGM-Prune enables only prune, and FastTGM enables both.

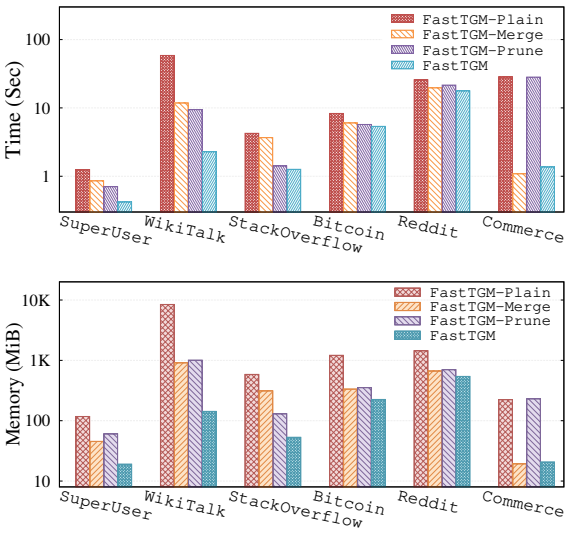
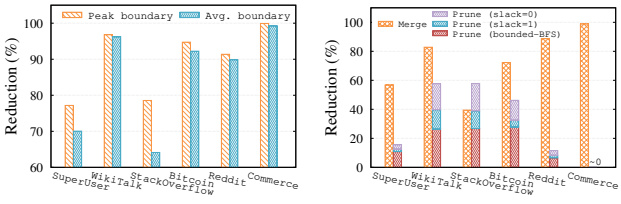


Figure 9: Ablation study of merge and prune effects on runtime and memory.



(a) Merge cuts state counts. (b) Merge and prune cut *take* calls.

Figure 10: Mechanism-level impact of merge and prune on state counts and take calls.

End-to-end contribution of merge and prune. Figure 9 shows that the two mechanisms are largely complementary. FastTGM-Plain is consistently the slowest and most memory-hungry variant. Enabling only one mechanism already cuts cost substantially, but the full system remains best or near-best overall: FastTGM gives the lowest runtime and the lowest memory on all datasets. The main exception is `Commerce`, where FastTGM-Merge is slightly better than the full system, indicating that this dataset is dominated by duplicate-state compression and leaves little additional pruning opportunity. Across the other datasets, prune contributes most of the standalone runtime gain on `SuperUser`, `WikiTalk`, and `StackOverflow`, whereas merge is the dominant source of memory reduction.

Mechanism view. Figure 10 provides a mechanism-level view of these gains. The left panel shows that merge directly shrinks the state load carried across decision boundaries: peak boundary state counts drop by 77–100%, and average boundary load by 64–99%. In terms of Theorem 2, this compresses the bucket sizes B_i and hence the $O(B_i \log B_i)$ sorting cost at each boundary. The right panel tracks reduction in state-extension (*take*) calls ($\sum_i T_i$ in the same bound). Merge alone already removes 39–99% of take calls, and prune adds a further dataset-dependent reduction. Most prune-side savings come from the bounded-BFS stage, while the slack-0 and slack-1 checks provide additional early cuts on datasets such as `WikiTalk` and `StackOverflow`. On `Commerce`, the prune-side bars are almost zero, which matches the end-to-end result that merge already captures nearly all available savings there.

7 Related Work

Temporal motif matching and counting. Temporal motifs extend network motifs to settings where edge order and time windows matter [15, 24, 38], with modeling choices and evaluation pitfalls surveyed in later work [19, 30]. A first line of algorithms treats a motif as a temporal query and accelerates single-pattern matching through ordering constraints, candidate filtering, and temporal consistency checks. Representative approaches include chronological edge-driven matching [32], temporal subgraph isomorphism [21, 27, 33, 39, 43], and temporal-constraint matching frameworks [10, 23, 25]. A second line targets scalable counting through exact temporal motif counters [9, 28, 44, 51], GPU execution [49], or sampling-based estimators [29, 36, 47]. FastTGM differs in problem shape: it computes exact, per-seed temporal motif vectors for all motif classes in one shared pass, rather than evaluating one query pattern or one global aggregate at a time.

Static graph mining and state sharing. General graph mining systems already exploit ordering, duplicate elimination, pruning, parallel execution, and CPU/GPU optimization [4, 5, 11, 12, 18, 40, 45]. In static graph enumeration, frontier-based search and decision-diagram compilation merge states sharing the same active frontier [17, 20, 41, 42, 48]. The temporal setting imposes a fundamentally different equivalence regime: two partial realizations with the same projected graph can have different continuation counts, because the processing order is dictated by motif chronology and realized vertices expire from future visibility as the suffix advances.

Risk-oriented pattern mining. Graph-based risk patterns are valued for interpretability in industrial fraud and risk management, but mining them at scale on large attributed graphs remains challenging [16, 31, 35, 37]. GraphRPM proposes a distributed framework for mining and evaluating risky subgraph patterns on industrial-scale attributed graphs [46]. Compared with static or aggregated pattern-mining pipelines, temporal-motif-based approaches can encode fine-grained interaction order and time locality, which are important in risk workflows [3, 6, 30, 38]. FastTGM addresses the feature-extraction bottleneck in this workflow by producing exact seed-rooted temporal motif vectors for downstream pattern selection, scoring, and evaluation.

8 Conclusion

We presented FastTGM, a compressed counting framework for exact seed-rooted temporal motif counting. Unlike prior methods that track every concrete partial realization separately, FastTGM introduces temporal future-equivalent state compression (ETFA), which collapses indistinguishable histories into compressed states with multiplicity counters, and slack-aware completion pruning, which eliminates infeasible continuation states with the pruning gain amplified by ETFA's compression. Together, these mechanisms replace occurrence-by-occurrence search with equivalence-class-level counting. Experiments confirm that FastTGM scales robustly to larger motif sizes where all baselines degrade or time out, and an industrial case study demonstrates that temporal motif features support practical risk-pattern discovery and early fraud detection.

References

[1] Rakesh Agrawal and Ramakrishnan Srikant. 1995. Mining Sequential Patterns. In *Proceedings of the Eleventh International Conference on Data Engineering (ICDE)*. 3–14. doi:10.1109/ICDE.1995.380415

[2] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.

[3] Jovan Blanuša, Maximo Cravero Baraja, Andreea Anghel, Luc Von Niederhäusern, Erik Altman, Haris Pozidis, and Kubilay Atasu. 2024. Graph Feature Preprocessor: Real-time Subgraph-based Feature Extraction for Financial Crime Detection. In *Proceedings of the 5th ACM International Conference on AI in Finance*. 222–230.

[4] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, Loc Hoang, and Keshav Pingali. 2021. Sandslash: A Two-Level Framework for Efficient Graph Pattern Mining. In *Proceedings of the ACM International Conference on Supercomputing (ICS '21)*. doi:10.1145/3447818.3460359

[5] Xuhao Chen, Roshan Dathathri, Gurbinder Gill, and Keshav Pingali. 2020. Pangolin: An efficient and flexible graph mining system on cpu and gpu. *Proceedings of the VLDB Endowment* 13, 8 (2020), 1190–1205.

[6] Yuhang Chen, Jiaxin Jiang, Shixuan Sun, Bingsheng He, and Min Chen. 2024. Rush: Real-time burst subgraph detection in dynamic graphs. *Proceedings of the VLDB Endowment* 17, 11 (2024), 3657–3665.

[7] Mohammed Elseidy, Ehab Abdelhamid, Spiros Skiadopoulos, and Panos Kalnis. 2014. Grami: Frequent subgraph and pattern mining in a single large graph. *Proceedings of the VLDB Endowment* 7, 7 (2014), 517–528.

[8] Philippe Fournier-Viger, Cheng-Wei Wu, and Vincent S Tseng. 2013. Mining maximal sequential patterns without candidate maintenance. In *International Conference on Advanced Data Mining and Applications*. Springer, 169–180.

[9] Zhongqiang Gao, Chuanqi Cheng, Yanwei Yu, Lei Cao, Chao Huang, and Junyu Dong. 2022. Scalable Motif Counting for Large-Scale Temporal Graphs. In *2022 IEEE 38th International Conference on Data Engineering (ICDE)*. 2657–2669. doi:10.1109/ICDE53745.2022.00244

[10] Roberto Grasso, Giovanni Micale, Alfredo Ferro, and Alfredo Pulvirenti. 2022. Modit: motif discovery in temporal networks. *Frontiers in big Data* 4 (2022), 806014.

[11] Wentian Guo, Yuchen Li, Mo Sha, Bingsheng He, Xiaokui Xiao, and Kian-Lee Tan. 2020. GPU-Accelerated Subgraph Enumeration on Partitioned Graphs. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (SIGMOD '20)*. 1067–1082. doi:10.1145/3318464.3389699

[12] Wentian Guo, Yuchen Li, and Kian-Lee Tan. 2022. Exploiting Reuse for GPU Subgraph Enumeration. *IEEE Transactions on Knowledge and Data Engineering* 34, 9 (2022), 4231–4244. doi:10.1109/TKDE.2020.3035564

[13] Jiawei Han, Jian Pei, Behzad Mortazavi-Asl, Helen Pinto, Qiming Chen, Umeshwar Dayal, and Meichun Hsu. 2001. Prefixspan: Mining sequential patterns efficiently by prefix-projected pattern growth. In *proceedings of the 17th international conference on data engineering*. IEEE Piscataway, NJ, USA, 215–224.

[14] Jack Hessel, Chenhao Tan, and Lillian Lee. 2016. Science, askscience, and badscience: On the coexistence of highly related communities. In *Proceedings of the international AAAI conference on web and social media*, Vol. 10. 171–180.

[15] Petter Holme and Jari Saramäki. 2012. Temporal networks. *Physics Reports* 519, 3 (2012), 97–125.

[16] Bryan Hooi, Kijung Shin, Alex Beutel, Christos Faloutsos, and Nitesh V. Chawla. 2016. Fraudar: Bounding graph fraud in the face of camouflage. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*. 895–904.

[17] Takeru Inoue, Hiroaki Iwashita, Jun Kawahara, and Shin-ichi Minato. 2016. Graphillion: Software Library for Very Large Sets of Labeled Graphs. *International Journal on Software Tools for Technology Transfer* 18 (2016), 57–66. doi:10.1007/s10009-014-0352-z

[18] Kasra Jamshidi, Rakesh Mahadasa, and Keval Vora. 2020. Peregrine: A Pattern-Aware Graph Mining System. In *Proceedings of the Fifteenth European Conference on Computer Systems (EuroSys '20)*. Article 13, 16 pages. doi:10.1145/3342195.3387548

[19] Ali Jazayeri and Christopher C Yang. 2020. Motif discovery algorithms in static and temporal networks: A survey. *Journal of Complex Networks* 8, 4 (2020), cnaa031.

[20] Jun Kawahara, Takeru Inoue, Hiroaki Iwashita, and Shin-ichi Minato. 2017. Frontier-Based Search for Enumerating All Constrained Subgraphs with Compressed Representation. *IEICE Transactions on Fundamentals of Electronics, Communications and Computer Sciences* E100.A, 9 (2017), 1773–1784. doi:10.1587/transfun.E100.A.1773

[21] Kyoungmin Kim, In Seo, Wook-Shin Han, Jeong-Hoon Lee, Sungpack Hong, Hassan Chafi, Hyungyu Shin, and Geonhwa Jeong. 2018. Turboflux: A fast continuous subgraph matching system for streaming graph data. In *Proceedings of the 2018 international conference on management of data*. 411–426.

[22] Dániel Kondor, István Csabai, János Szüle, Márton Pósfai, and Gábor Vattay. 2014. Inferring the interplay between network structure and market effects in bitcoin. *New Journal of Physics* 16, 12 (2014), 125003.

[23] Chrysanthi Kosyfaki, Nikos Mamoulis, Evaggelia Pitoura, and Panayiotis Tsaparas. 2019. Flow Motifs in Interaction Networks. In *Proceedings of the 22nd International Conference on Extending Database Technology (EDBT)*. 241–252.

[24] Lauri Kovanen, Márton Karsai, Kimmo Kaski, János Kertész, and Jari Saramäki. 2011. Temporal motifs in time-dependent networks. *Journal of Statistical Mechanics: Theory and Experiment* 2011, 11 (2011), P11005. doi:10.1088/1742-5468/2011/11/P11005

[25] Xiaoyu Leng, Guang Zeng, Hongchao Qin, Longlong Lin, and Rong-Hua Li. 2025. On Temporal-Constraint Subgraph Matching. In *Proceedings of the 41st IEEE International Conference on Data Engineering (ICDE)*. 2493–2506.

[26] Jure Leskovec and Andrej Krevl. 2014. SNAP Datasets: Stanford Large Network Dataset Collection. https://snap.stanford.edu/data/.

[27] Faming Li and Zhaonian Zou. 2021. Subgraph matching on temporal graphs. *Information Sciences* 578 (2021), 539–558.

[28] Jiantao Li, Jianpeng Qi, Yueling Huang, Lei Cao, Yanwei Yu, and Junyu Dong. 2024. MoTTo: Scalable Motif Counting with Time-aware Topology Constraint for Large-Scale Temporal Graphs. In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management (CIKM)*. 1195–1204.

[29] Paul Liu, Austin R. Benson, and Moses Charikar. 2019. Sampling Methods for Counting Temporal Motifs. In *Proceedings of the 12th ACM International Conference on Web Search and Data Mining (WSDM)*. 294–302.

[30] Paul Liu, Valeria Guarrasi, and Ahmet Erdem Sariyüce. 2023. Temporal Network Motifs: Models, Limitations, Evaluation. *IEEE Transactions on Knowledge and Data Engineering* 35, 7 (2023), 7058–7071. doi:10.1109/TKDE.2021.3128251

[31] Yice Luo, Guannan Wang, Yongchao Liu, Jiaxin Yue, Weihong Cheng, and Binjie Fei. 2023. FAF: a risk detection framework on industry-scale graphs. In *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management*. 4717–4723.

[32] Patrick Mackey, Katherine Porterfield, Erin Fitzhenry, Sutanay Choudhury, and George Chin. 2018. A Chronological Edge-Driven Approach to Temporal Subgraph Isomorphism. In *2018 IEEE International Conference on Big Data (Big Data)*. 3972–3979. doi:10.1109/BigData.2018.8622100

[33] Giovanni Micale, Giorgio Locicero, Alfredo Pulvirenti, and Alfredo Ferro. 2021. TemporalRI: subgraph isomorphism in temporal networks with multiple contacts. *Applied Network Science* 6, 1 (2021), 55.

[34] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network motifs: simple building blocks of complex networks. *Science* 298, 5594 (2002), 824–827. doi:10.1126/science.298.5594.824

[35] Mohammad H. Namaki, Yinghui Wu, Qi Song, Peng Lin, and Tingjian Ge. 2017. Discovering Graph Temporal Association Rules. In *Proceedings of the 26th ACM International Conference on Information and Knowledge Management (CIKM)*. 1697–1706.

[36] Yunjie Pan, Omkar Bhalerao, C. Seshadhri, and Nishil Talati. 2024. Accurate and Fast Estimation of Temporal Motifs using Path Sampling. arXiv preprint arXiv:2409.08975. doi:10.48550/arXiv.2409.08975 Accepted in ICDM'24 (per arXiv record).

[37] Shashank Pandit, Duen Horng Chau, Samuel Wang, and Christos Faloutsos. 2007. NetProbe: a fast and scalable system for fraud detection in online auction networks. In *Proceedings of the 16th International Conference on World Wide Web (WWW)*. 201–210.

[38] Ashwin Paranjape, Austin R Benson, and Jure Leskovec. 2017. Motifs in temporal networks. In *Proceedings of the tenth ACM international conference on web search and data mining*. 601–610.

[39] Konstantinos Semertzidis and Evaggelia Pitoura. 2020. A hybrid approach to temporal pattern matching. In *2020 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM)*. IEEE, 384–388.

[40] Tianhui Shi, Mingshu Zhai, Yi Xu, and Jidong Zhai. 2020. GraphPi: High Performance Graph Pattern Matching through Effective Redundancy Elimination. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*.

[41] Teruji Sugaya, Masaaki Nishino, Norihito Yasuda, and Shin-ichi Minato. 2018. Fast Compilation of Graph Substructures for Counting and Enumeration. *Behaviormetrika* 45 (2018), 423–450. doi:10.1007/s41237-018-0056-x

[42] Teruji Sugaya, Masaaki Nishino, Norihito Yasuda, and Shin-ichi Minato. 2020. Tree Decomposition-based Approach for Compiling Independent Sets. *Journal of Information Processing* 28 (2020), 354–368. doi:10.2197/ipsjjip.28.354

[43] Shixuan Sun and Qiong Luo. 2022. Subgraph matching with effective matching order and indexing. *IEEE Transactions on Knowledge and Data Engineering* 34, 1 (2022), 491–505. doi:10.1109/TKDE.2020.2980257

[44] Xiaoli Sun, Yusong Tan, Qingbo Wu, Baozi Chen, and Changxiang Shen. 2019. TM-Miner: TFS-Based Algorithm for Mining Temporal Motifs in Large Temporal Network. *IEEE Access* 7 (2019), 49778–49789.

[45] Carlos HC Teixeira, Alexandre J Fonseca, Marco Serafini, Georgos Siganos, Mohammed J Zaki, and Ashraf Aboulnaga. 2015. Arabesque: a system for distributed graph mining. In *Proceedings of the 25th Symposium on Operating Systems Principles*. 425–440.

[46] Sheng Tian, Xintan Zeng, Yifei Hu, Baokun Wang, Yongchao Liu, Yue Jin, Changhua Meng, Chuntao Hong, Tianyi Zhang, and Weiqiang Wang. 2024. GraphRPM: Risk Pattern Mining on Industrial Large Attributed Graphs. arXiv:2411.06878 [cs.LG]

[47] Jingjing Wang, Yanhao Wang, Wenjun Jiang, Yuchen Li, and Kian-Lee Tan. 2020. Efficient Sampling Algorithms for Approximate Temporal Motif Counting.

In *Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM)*. 1505–1514.

[48] Norihito Yasuda, Teruji Sugaya, and Shin-Ichi Minato. 2017. Fast compilation of st paths on a graph for counting and enumeration. In *Advanced Methodologies for Bayesian Networks*. PMLR, 129–140.

[49] Yichao Yuan, Haojie Ye, Sanketh Vedula, Wynn Kaza, and Nishil Talati. 2023. Everest: GPU-Accelerated System For Mining Temporal Motifs. *Proceedings of the VLDB Endowment* 17, 2 (2023), 162–174.

[50] Mohammed J. Zaki. 2001. SPADE: An Efficient Algorithm for Mining Frequent Sequences. *Machine Learning* 42, 1–2 (2001), 31–60. doi:10.1023/A:1007652502315

[51] Zhihao Zhang, Jianpeng Qi, Lei Cao, Junyu Dong, and Yanwei Yu. 2025. Efficiently Counting Four-Node Motifs in Large-Scale Temporal Graphs. *VLDB Journal* 34, 4 (2025), Article 45 (24 pages).

Algorithm 3: eDFS: Incidence-Driven DFS

Input: temporal graph $\mathcal{T}$, seed u, motif size L, time window δ
Output: temporal motif vector $\mathbf{TMV}_u$

$\mathbf{TMV}_u \leftarrow \mathbf{0}$;
Seen $\leftarrow \emptyset$;
foreach *anchor edge* $e_0 \in out(u)$ **do**
  $P \leftarrow \{e_0\}$;
  ExpandDFS(P, e_0);
Procedure ExpandDFS(P, e_0)
  if $|P| = L$ **then**
    $h \leftarrow$ OccurrenceKey(P);
    if $h \notin$ Seen **then**
      Seen.insert(h);
      $\xi \leftarrow$ Encode(ChronologicalOrder(P));
      $\mathbf{TMV}_u[\xi] \leftarrow \mathbf{TMV}_u[\xi] + 1$;
    return;
  $C \leftarrow \{e \mid e \notin P,\ \tau(e_0) < \tau(e) \le \tau(e_0) + \delta,\ e$ is incident to an endpoint of some edge in $P\}$;
  foreach $e \in C$ **do**
    ExpandDFS$(P \cup \{e\}, e_0)$;
end

Algorithm 4: EComb: Chronological Combination

Input: temporal graph $\mathcal{T}$, seed u, motif size L, time window δ
Output: temporal motif vector $\mathbf{TMV}_u$

$\mathbf{TMV}_u \leftarrow \mathbf{0}$;
foreach *anchor edge* $e_0 \in out(u)$ **do**
  $\sigma(e_0) \leftarrow$ anchor-local chronological suffix of e_0;
  foreach $S \subseteq \sigma(e_0)$ *with* $|S| = L - 1$ **do**
    $P \leftarrow \{e_0\} \cup S$;
    if P *is connected* **then**
      $\xi \leftarrow$ Encode(ChronologicalOrder(P));
      $\mathbf{TMV}_u[\xi] \leftarrow \mathbf{TMV}_u[\xi] + 1$;

A Deferred Proofs

This appendix contains the proofs of the auxiliary ETFA lemmas omitted from the submission body.

A.1 Proof of Lemma 1

Proof. By definition of $\eta_i(P)$, every live frontier vertex has a future incidence at or after position $\eta_i(P)$, so none can expire earlier. Since no edge is selected on an exclude-only path, the partial realization and its canonical descriptor remain unchanged throughout the interval. Thus, all intermediate configurations are identical, and the only state update occurs when expiration is applied at $\eta_i(P)$. □

A.2 Proof of Lemma 3

Proof. Suppose bucket i contains entries

$$(\mathcal{A}, c_1), (\mathcal{A}, c_2), \ldots, (\mathcal{A}, c_t)$$

with the same ETFA state $\mathcal{A}$. By Lemma 2, every concrete history represented by $\mathcal{A}$ contributes the same continuation multiset, denoted by $\mathrm{Cont}(\mathcal{A})$. Therefore,

$$\biguplus_{\ell=1}^{t} c_\ell \cdot \mathrm{Cont}(\mathcal{A}) = \left(\sum_{\ell=1}^{t} c_\ell\right) \cdot \mathrm{Cont}(\mathcal{A}),$$

where $c \cdot \mathrm{Cont}(\mathcal{A})$ denotes the c-fold multiset sum of $\mathrm{Cont}(\mathcal{A})$. Hence replacing these entries by one entry with multiplicity $\sum_{\ell=1}^{t} c_\ell$ preserves the total accepting counts contributed by boundary i. □

B Constructed Baselines

This appendix gives the pseudocode for the three controlled baselines used in the experiments. All three are implemented in the same mining framework as FastTGM and share the same temporal graph representation, time-window semantics, anchor-local candidate construction, and canonical motif encoder whenever applicable. They differ in how they organize the search space: incidence-driven traversal, chronological combination, and chronological combination with necessary structural filtering. This makes the comparison isolate where the end-to-end cost comes from before adding future-equivalent state compression.

B.1 eDFS: Incidence-Driven DFS with Deduplication

eDFS is an incidence-driven traversal baseline. Starting from an anchor edge $e_0 \in out(u)$, it recursively adds temporal edges that are incident to the currently selected edge set and lie in the anchor window $(\tau(e_0), \tau(e_0) + \delta]$. Thus every intermediate prefix is connected by construction. Compared with chronological combination, the cost shifts from late connectivity checks to repeated construction of the same completed edge set under different incidence-driven expansion orders.

Deduplication. To avoid double counting, eDFS maintains a hash set over completed occurrence identities. The hash key is the concrete selected edge set, not the motif class. After a completed edge set passes this occurrence deduplication step, the edge set is ordered chronologically, encoded into its canonical motif class, and accumulated into $\mathbf{TMV}_u$. This introduces memory overhead proportional to the number of distinct completed occurrences discovered for the seed.

Comparison axis. eDFS isolates the behavior of connectivity-preserving expansion. It performs no work on disconnected prefixes, while exposing the overhead of expansion-order duplicates and occurrence-level deduplication.

B.2 EComb: Chronological Combination

EComb is the chronological-combination baseline. For each anchor edge e_0, it constructs the anchor-local suffix $\sigma(e_0)$ and enumerates every $(L-1)$-edge subset of this suffix. The anchor edge is fixed as the earliest edge, and the remaining selected edges are drawn from the strictly later suffix. Therefore each completed edge set is generated once under the anchor-local chronological representation, and no occurrence-level deduplication hash table is needed.

Connectivity and encoding. Unlike eDFS, EComb does not enforce connectivity during selection. Each selected set $P = \{e_0\} \cup S$ is tested for connectivity after all L edges have been chosen. If P is connected, its chronological edge sequence is encoded into a motif class and added to $\mathbf{TMV}_u$. The method therefore removes duplicate construction at the search-space level, but it may enumerate structurally invalid subsets before the final connectivity test.

Cost profile. Let $m_{e_0} = |\sigma(e_0)|$. The dominant term is the enumeration of $\binom{m_{e_0}}{L-1}$ subsets per anchor, followed by connectivity and

Algorithm 5: EComb+: Pruned Chronological Combination

Input: temporal graph $\mathcal{T}$, seed u, motif size L, time window δ
Output: temporal motif vector $\mathbf{TMV}_u$

$\mathbf{TMV}_u \leftarrow \mathbf{0}$;
foreach *anchor edge* $e_0 \in out(u)$ **do**
  $\sigma(e_0) = \langle e_1, \ldots, e_m \rangle \leftarrow$ anchor-local chronological suffix of e_0;
  PrunedComb($\{e_0\}, 1, \sigma(e_0)$);
Procedure PrunedComb(P, i, σ)
  if $|P| = L$ **then**
    if P *is connected* **then**
      $\xi \leftarrow$ Encode(ChronologicalOrder(P));
      $\mathbf{TMV}_u[\xi] \leftarrow \mathbf{TMV}_u[\xi] + 1$;
    return;
  $C \leftarrow \{j \mid i \le j \le |\sigma|\}$;
  $C \leftarrow$ SlackRoute(P, C, σ);
  $C \leftarrow$ LevelFilter(P, C, σ);
  foreach $j \in C$ *in increasing order* **do**
    PrunedComb($P \cup \{\sigma[j]\}, j + 1, \sigma$);
end

encoding work for each subset. Thus EComb isolates chronological duplicate elimination without structural filtering or state sharing.

B.3 EComb+: Pruned Chronological Combination

EComb+ keeps the same chronological search space as EComb but implements it recursively so that necessary structural conditions can be checked before selecting the next suffix edge. It still enumerates concrete prefixes one by one; multiplicities are not aggregated across future-equivalent prefixes.

Connectivity-budget routing. For a partial edge set P, EComb+ computes the connected components of the static graph induced by P and classifies each future suffix candidate by how it affects these components. A *bridge* connects two current components, an *attach* edge touches the current prefix without reducing the component count, and a *spawn* edge introduces a new disconnected component. If the remaining edge budget exactly matches the current connectivity debt, only bridges can appear in any connected completion. If there is one extra edge beyond the debt, spawn edges are still impossible because they would create an additional component that cannot be reconnected within the remaining budget.

Level filtering. EComb+ also applies a seed-distance filter. Each candidate edge is assigned a level according to its distance from the seed in the static projection. A connected length-L seed-rooted motif cannot contain too many high-level edges, because such edges require supporting edges to connect back to the seed side. The level filter therefore removes a candidate when adding it would violate this necessary size-L budget.

Comparison axis. EComb+ isolates the effect of necessary structural filtering inside the chronological search space. Compared with EComb, it removes candidate branches that cannot fit a connected length-L completion. Compared with FastTGM, it keeps concrete-prefix execution: filtering is applied before expansion, but equivalent continuation states are not merged into multiplicity-carrying ETFA states.